

Branching Chat Workspace

One-page repo-based summary

WHAT IT IS

This is a runnable multi-service "Kuse" style branching research canvas app. It combines a Next.js canvas client, a GraphQL gateway, and shared schemas to generate, stream, and persist graph-based conversations.

WHO IT IS FOR

Primary persona: users who need to break complex questions into structured, visual branching graphs for research/planning workflows.

Specific job title/industry targeting: **Not found in repo.**

WHAT IT DOES

- Provides a multi-panel workspace UI (canvas viewport, assistant panel, toolbar, node palette, document drawer, timeline history panel).
- Loads canvas graph data via GraphQL `workspaceGraph` query.
- Lets users add nodes and connect nodes through GraphQL mutations (`addNode` , `connectNodes`).
- Starts AI-driven graph generation with `startConversation` .
- Streams incremental graph/status updates with GraphQL subscription `conversationProgress` over WebSocket.
- Persists workspace graphs to PostgreSQL table `canvas_graphs` .
- Includes knowledge-base task flows (create/publish KB, seed/import URL, status tracking, and file import proxy to task service).

HOW IT WORKS

`apps/web` `packages/server` `packages/shared`
`backend`

- 1) Frontend (Next.js):
 - Uses `graphql-request` + `React Query` for queries/mutations
 - Uses `graphql-ws` for subscription streaming
- 2) Gateway (Express + Apollo):
 - Exposes `/graphql` (HTTP + WS)
 - Resolvers call `ConversationStore` + KB task service helpers
- 3) Conversation engine:
 - `startConversation` -> `ConversationStore` -> `BusinessLangGraphService`
 - Emits `graph/appended`, `graph/diff`, status events via PubSub
- 4) Data and services:
 - Canvas graphs saved/loaded in PostgreSQL (`canvas_graphs`)
 - KB task status kept in `TaskEventStore` (in-memory projection)
 - Gateway proxies KB file import to backend task service (`:4001`)
 - Supabase and Dify integration code exists; usage depends on config/route path

HOW TO RUN (MINIMAL)

1. From repo root: `pnpm install`
2. Create `packages/server/.env` with at least `DATABASE_URL` (and `KB_TASK_SERVICE_URL` if using KB task APIs).
3. Start GraphQL gateway: `pnpm dev:server` (`http://localhost:4000/graphql`)
4. Start frontend: `pnpm dev:web` (`http://localhost:3000`)
5. Optional for KB import/indexing flows: run standalone backend service in `backend/` .

Evidence sources: README.md, package.json, apps/web app and workspace feature files, packages/server GraphQL schema/resolvers/context/store/persistence/routes, packages/shared schemas, backend/README.md.